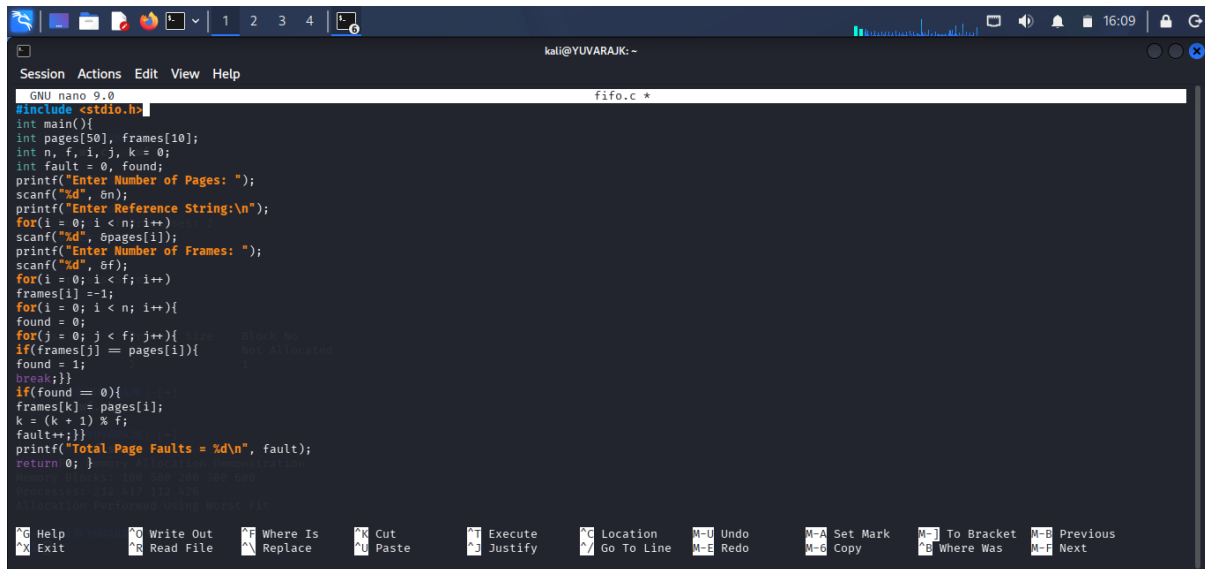


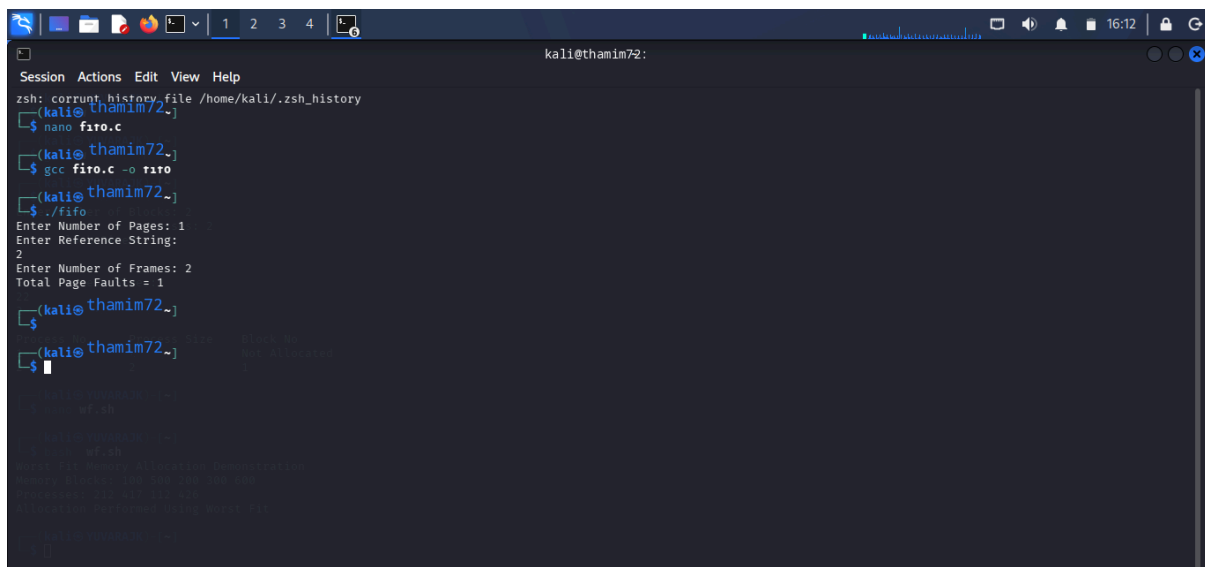
C PROGRAMMING : FIFO



```
GNU nano 9.0 fifo.c *
#include <stdio.h>

int main(){
    int pages[50], frames[10];
    int n, f, i, j, k = 0;
    int fault = 0, found;
    printf("Enter Number of Pages: ");
    scanf("%d", &n);
    printf("Enter Reference String:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);
    printf("Enter Number of Frames: ");
    scanf("%d", &f);
    for(i = 0; i < f; i++)
        frames[i] = -1;
    for(i = 0; i < n; i++){
        found = 0;
        for(j = 0; j < f; j++){
            if(frames[j] == pages[i]){
                found = 1;
                break;
            }
        }
        if(found == 0){
            frames[k] = pages[i];
            k = (k + 1) % f;
            fault++;
        }
    }
    printf("Total Page Faults = %d\n", fault);
    return 0; }
```

OUTPUT :



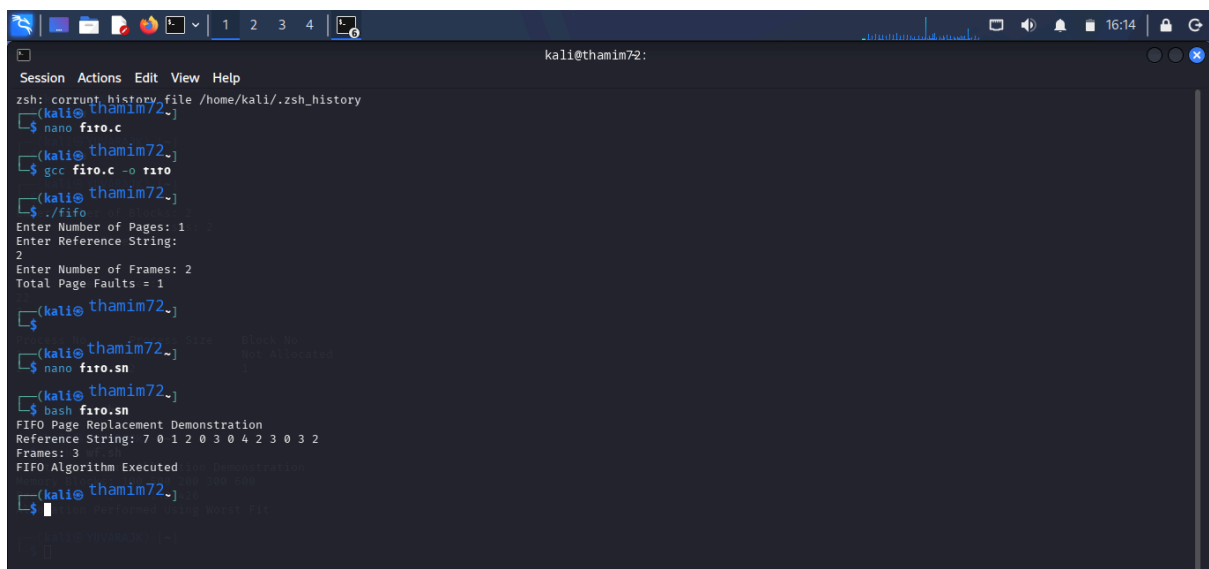
```
kali@thamim72:~$ zsh: corrupt history file /home/kali/.zsh_history
(kali@thamim72~)
(kali@thamim72~) $ nano fifo.c
(kali@thamim72~) $ gcc fifo.c -o fifo
(kali@thamim72~) $ ./fifo
Enter Number of Pages: 1
Enter Reference String:
2
Enter Number of Frames: 2
Total Page Faults = 1
(kali@thamim72~) $
```

SHELL PROGRAMMING : FIFO



```
GNU nano 9.0 fifo.sh
#!/bin/bash
echo "FIFO Page Replacement Demonstration"
pages=(7 0 1 2 0 3 0 4 2 3 0 3 2)
frames=3
echo "Reference String: ${pages[@]}"
echo "Frames: $frames"
echo "FIFO Algorithm Executed"
```

OUTPUT :



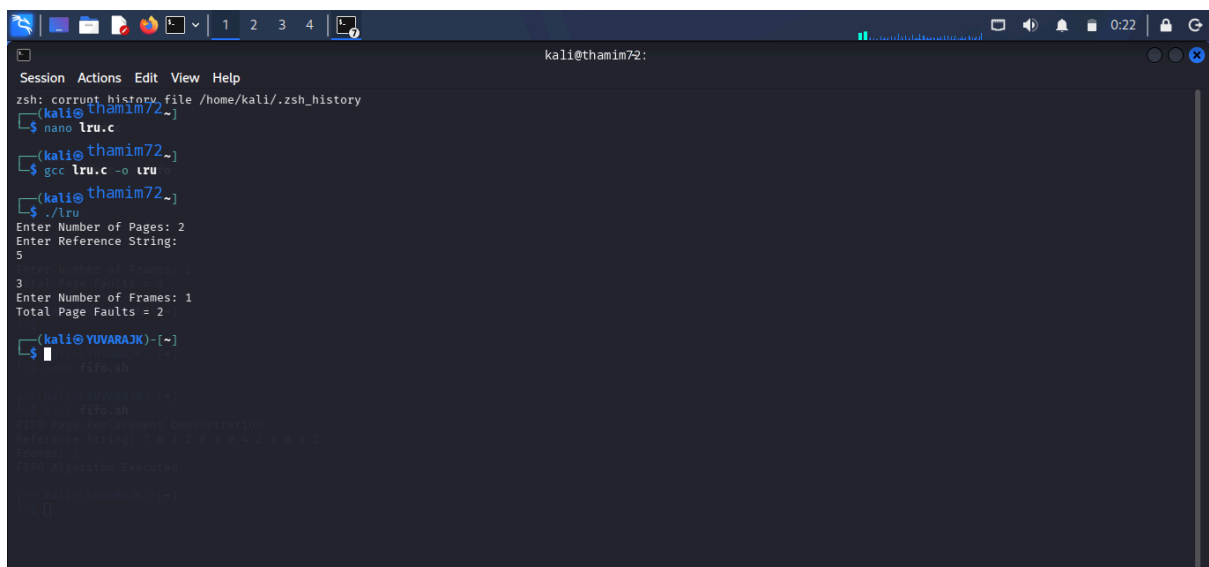
```
kali@thamim72:~
zsh: current_history file /home/kali/.zsh_history
(kali@thamim72~)
$ nano firo.c
(kali@thamim72~)
$ gcc firo.c -o firo
(kali@thamim72~)
$ ./firo
Enter Number of Pages: 1
Enter Reference String:
2
Enter Number of Frames: 2
Total Page Faults = 1
(kali@thamim72~)
$ nano firo.sn
(kali@thamim72~)
$ bash firo.sn
FIFO Page Replacement Demonstration
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2
Frames: 3
FIFO Algorithm Executed
(kali@thamim72~)
$
```

C PROGRAMMING :LRU



```
GNU nano 9.0 lru.c *
#include <stdio.h>
int main() {
    int pages[50], frames[10], time[10];
    int n, f, i, j;
    int fault = 0, count = 0;
    int found, pos, min;
    printf("Enter Number of Pages: ");scanf("%d", &n);
    printf("Enter Reference String:\n");
    for(i = 0; i < n; i++)scanf("%d", &pages[i]);
    printf("Enter Number of Frames: ");scanf("%d", &f);
    for(i = 0; i < f; i++)frames[i] = -1;
    for(i = 0; i < n; i++){found = 0;
    for(j = 0; j < f; j++){
    if(frames[j] == pages[i]){count++;time[j] = count;
    found = 1;break;}}
    if(found == 0){
    min = time[0];
    pos = 0;
    for(j = 0; j < f; j++){
    if(frames[j] == -1){pos = j;break;}
    if(time[j] < min){
    min = time[j];
    pos = j;}}
    frames[pos] = pages[i];
    count++;
    time[pos] = count;
    fault++;}}
    printf("Total Page Faults = %d\n", fault);
    return 0;
}
```

OUTPUT :



```
kali@thamim72:~
zsh: corrupt history file /home/kali/.zsh_history
(kali@thamim72~)
$ nano lru.c
(kali@thamim72~)
$ gcc lru.c -o lru
(kali@thamim72~)
$ ./lru
Enter Number of Pages: 2
Enter Reference String:
5
Enter Number of Frames: 3
Enter Number of Frames: 1
Total Page Faults = 2
(kali@YUVARAJK)-[~]
$
```

SHELL PRROGRAMMING : LRU

The screenshot shows a Kali Linux terminal window with the following content:

```

kali@YUVARAJIK: ~
Session Actions Edit View Help
GNU nano 9.0 lru.sh
#!/bin/bash
echo "LRU Page Replacement Demonstration"
pages=(7 0 1 2 0 3 0 4 2 3 0 3 2)
echo "Reference String: ${pages[@]}"
echo "LRU Algorithm Executed"

# LRU Page Replacement Demo
# This script demonstrates the LRU (Least Recently Used) page replacement algorithm.
# It takes a sequence of page requests and replaces the least recently used page
# when a new page is requested that is not currently in memory.

# Parameters
# - pages: An array of page numbers to be requested.

# Variables
# - pages: Array of page numbers.
# - size: Number of frames (pages that can be held in memory).
# - lru_list: A list to keep track of the pages in memory, ordered by their
#           last access time (least recent at the front).

# Function to initialize the LRU list
init_lru_list() {
    lru_list=()
}

# Function to add a page to the LRU list
add_to_lru_list() {
    local page=$1
    if [[ ! " ${lru_list[*]} " =~ " $page " ]]; then
        lru_list+=($page)
    fi
}

# Function to remove the least recently used page from the LRU list
remove_lru() {
    lru_list=( ${lru_list[@:1]} )
}

# Main execution
init_lru_list
for page in "${pages[@]}; do
    add_to_lru_list $page
    if [[ ${#lru_list[@]} -gt $size ]]; then
        remove_lru
    fi
done

echo "LRU Page Replacement Demonstration"
echo "Reference String: ${pages[@]}"
echo "LRU Algorithm Executed"

```

The terminal window has a menu bar with "Session", "Actions", "Edit", "View", and "Help". The status bar at the bottom contains the following shortcuts:

- Ctrl-H Help
- Ctrl-W Write Out
- Ctrl-Alt-W Where Is
- Ctrl-U Cut
- Ctrl-E Execute
- Ctrl-J Justify
- Ctrl-L Location
- Ctrl-G Go To Line
- Ctrl-M Undo
- Ctrl-E Redo
- Ctrl-M-A Set Mark
- Ctrl-M-B Copy
- Ctrl-M-] To Bracket
- Ctrl-M-^ Where Was
- Ctrl-M-; Previous
- Ctrl-M-~ Next

OUTPUT :

```

kali@thamim72:~$ zsh: corrupt history file /home/kali/.zsh_history
kali@thamim72:~$ nano lru.c
kali@thamim72:~$ gcc lru.c -o lru
kali@thamim72:~$ ./lru
Enter Number of Pages: 2
Enter Reference String:
5

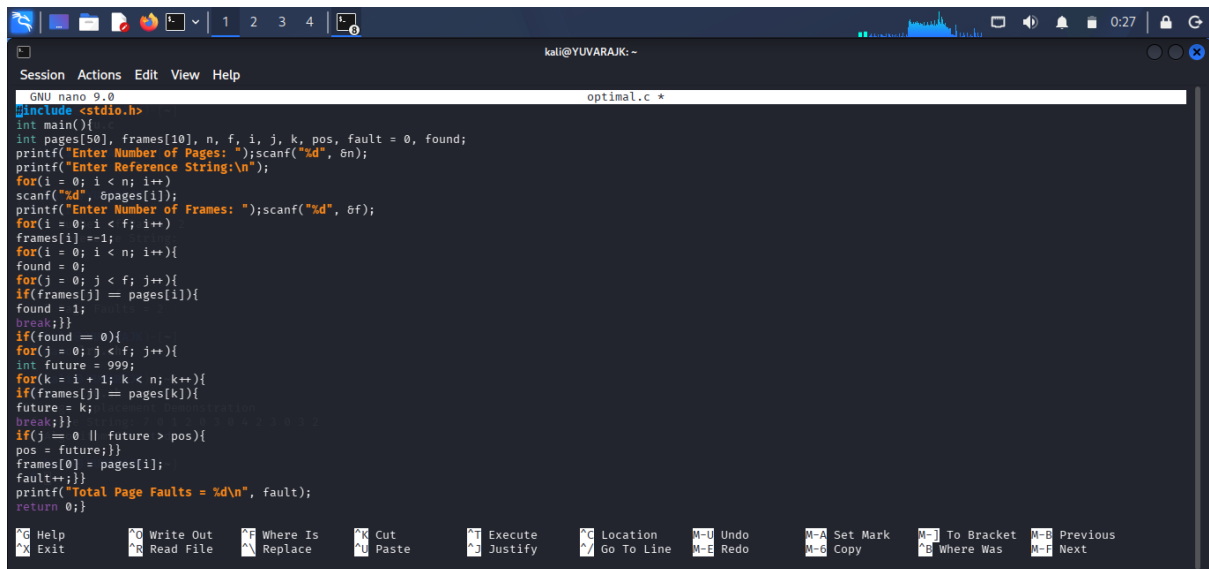
3
Enter Number of Frames: 1
Total Page Faults = 2

kali@thamim72:~$ nano lru.sn
kali@thamim72:~$ bash lru.sn
LRU Page Replacement Demonstration
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2
LRU Algorithm Executed

kali@thamim72:~$

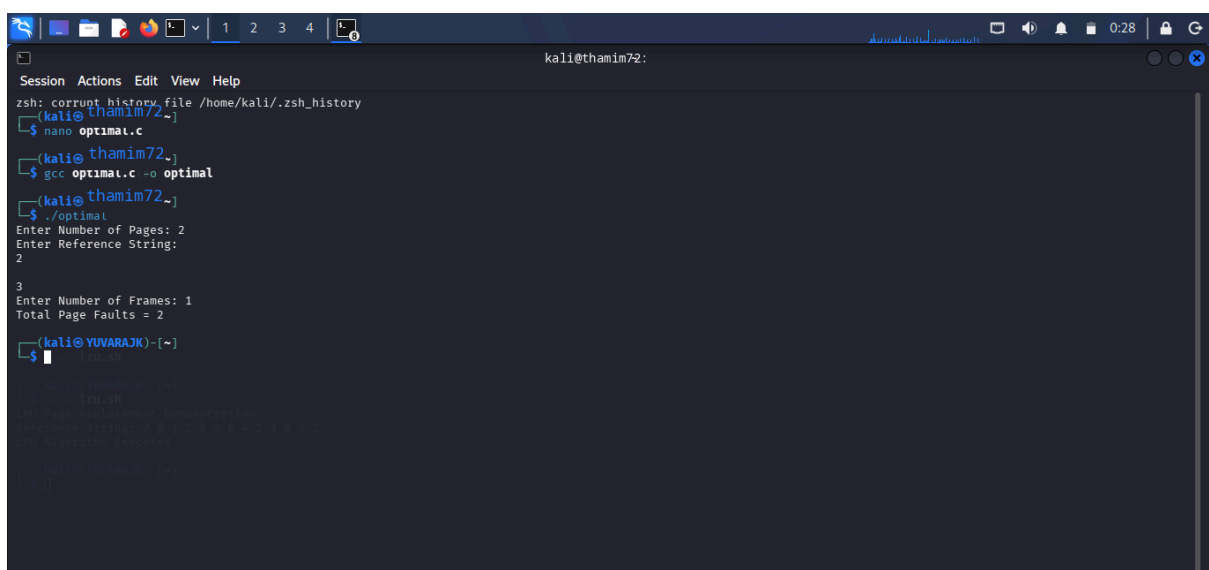
```

C PROGRAMMING :OPTIMAL PAGE



```
GNU nano 9.0 optimal.c *
#include <stdio.h>
int main(){
    int pages[50], frames[10], n, f, i, j, k, pos, fault = 0, found;
    printf("Enter Number of Pages: ");scanf("%d", &n);
    printf("Enter Reference String:\n");
    for(i = 0; i < n; i++){
        scanf("%d", &pages[i]);
    }
    printf("Enter Number of Frames: ");scanf("%d", &f);
    for(i = 0; i < f; i++){
        frames[i] = -1;
    }
    for(i = 0; i < n; i++){
        found = 0;
        for(j = 0; j < f; j++){
            if(frames[j] == pages[i]){
                found = 1;
                break;
            }
        }
        if(found == 0){
            for(j = 0; j < f; j++){
                int future = 999;
                for(k = i + 1; k < n; k++){
                    if(frames[j] == pages[k]){
                        future = k;
                        break;
                    }
                }
                if(j == 0 || future < pos){
                    pos = future;
                }
                frames[j] = pages[i];
                fault++;
            }
        }
        printf("Total Page Faults = %d\n", fault);
    }
    return 0;
}
```

OUTPUT :



```
kali@thamim72:~$ zsh: corrupt history file /home/kali/.zsh_history
(kali@thamim72~)
(kali@thamim72~)$ nano optimal.c
(kali@thamim72~)$ gcc optimal.c -o optimal
(kali@thamim72~)$ ./optimal
Enter Number of Pages: 2
Enter Reference String:
2
3
Enter Number of Frames: 1
Total Page Faults = 2
(kali@thamim72~)$
```

SHELL PROGRAMMING :

The screenshot shows a Kali Linux terminal window with the title 'kali@YUVARAJK: ~'. Inside the terminal, the nano text editor is open, editing a file named 'optimal.sh'. The editor's interface includes a top menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The main editing area contains the following code:

```
#!/bin/bash
echo "Optimal Page Replacement Demonstration"
pages=(7 0 1 2 0 3 0 4 2 3 0 3 2)
echo "Reference String: ${pages[@]}"
echo "Optimal Algorithm Executed"
```

The bottom status bar of the nano editor displays 'Read 5 lines'. Below the editor, a row of keyboard shortcuts is visible, including 'H Help', 'W Write Out', 'W Where Is', 'C Cut', 'E Execute', 'J Justify', 'L Location', 'G Go To Line', 'U Undo', 'M-M Undo Redo', 'S Set Mark', 'T To Bracket', 'P Previous', 'X Exit', 'R Read File', 'A Replace', 'U Paste', 'B Where Was', and 'N Next'.

OUTPUT :

```

kali@thamim72:
Session Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history
(kali@thamim72~)
$ nano optimal.c
(kali@thamim72~)
$ gcc optimal.c -o optimal
(kali@thamim72~)
$ ./optimal
Enter Number of Pages: 2
Enter Reference String:
2

3
Enter Number of Frames: 1
Total Page Faults = 2
(kali@thamim72~)
$ nano optimal.sn
(kali@thamim72~)
$ bash optimal.sn
Optimal Page Replacement Demonstration
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2
Optimal Algorithm Executed
(kali@thamim72~)
$

```